Kipi System: Data Flow Architecture

How 25 agents communicate without sharing context — a clean, file-based bus architecture built for reliability and scale.

ARCHITECTURE

The Bus Pattern

How It Works

Agents never communicate through shared conversation context. Each agent operates independently — reading input files from a shared `bus/` directory, executing its task, and writing output files back to the same location.

The bus is organized by date: `bus/2026-03-19/`

Files are overwritten on each run. **Every day starts completely clean.**

Read

Agent reads input `.json` files from `bus/date/`

Execute

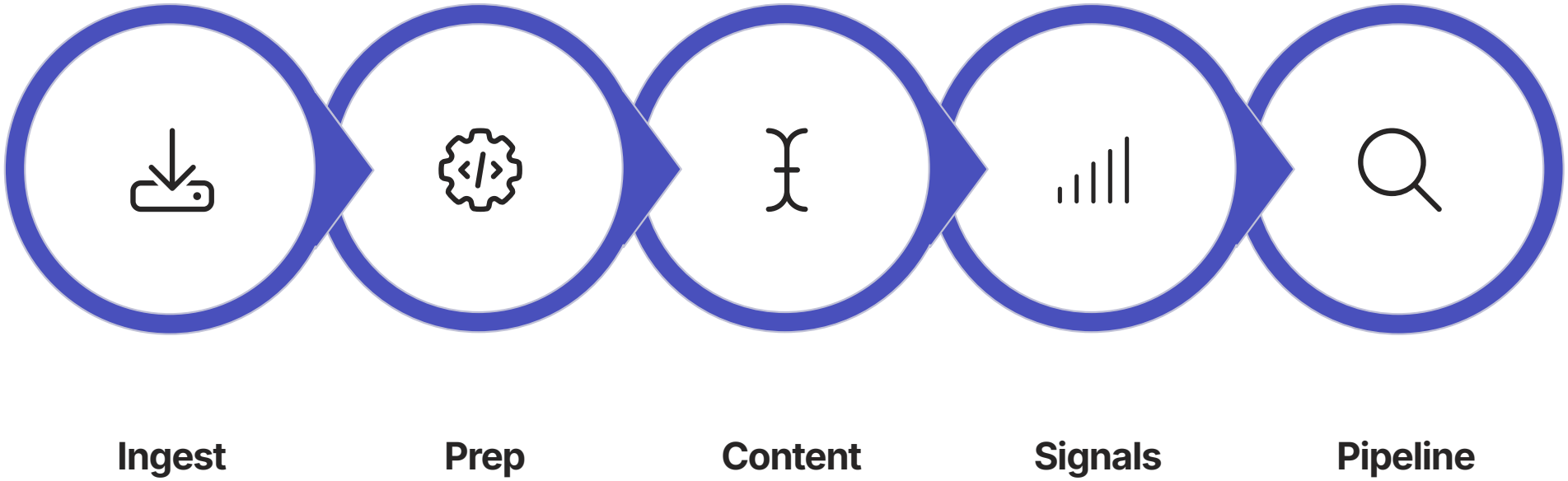
Agent performs its isolated task independently

Write

Agent writes output `.json` files back to bus

DATA FLOW

Bus File Map



Each phase reads upstream outputs and writes downstream inputs — a strict one-way data flow with no back-channels between agents.

RESILIENCE

Fallback Chains

The system degrades gracefully and automatically — no human intervention required until a hard stop is reached.

Apify Fails

→ Automatically falls back to `Chrome browser scraping`. No interruption to pipeline.

Notion API Fails

→ Falls back to `curl with token`. Data retrieval continues uninterrupted.

Gmail / Calendar Down

→ Skips all dependent steps, logs the gap, and **notes the skip in the briefing**. Pipeline continues.

Chrome Fails

→ **STOP.** No automated fallback exists. System reports failure and requires manual resolution.

SAFETY

Guardrail Architecture

1

Layer 3 — Audit Harness

Post-build verification, step completion log, skipped step detection

2

Layer 2 — CLAUDE.md Rules

Loaded every session. Enforces voice, positioning, and actionability standards

3

Layer 1 — Hooks

Token-guard on every tool call · Tool-counter on expensive ops · Session-start auto-loader · Diagnose-tools before commands

Three nested layers ensure every agent call is guarded, every session is consistently initialized, and every build is independently verified.

OUTPUT

The Schedule JSON

What the Synthesis Agent Produces

One JSON file conforming to a strict schema. Sections ordered by friction level. A build script converts this to HTML — **the user never sees raw JSON.**

Every Item Includes

→ **Copy-paste text**

Ready-to-send, no editing required

→ **Next physical action**

Specific, unambiguous next step

→ **Energy tag + time estimate**

Helps with scheduling and prioritization

→ **Direct links**

No hunting — everything surfaces inline

COMPATIBILITY

Platform Support Matrix

Capability	Claude Code	Cowork	Claude.ai Projects
Sub-agents	✅ Full	✅ Full	❌ None
Bash scripts	✅ Full	❌ Lost	❌ None
Hooks	✅ Full	❌ Lost	❌ None
MCP	✅ Full	✅ Full	❌ None
CLAUDE.md auto-load	✅ Full	❌ Lost	❌ Manual only
Skills / Files	✅ Full	✅ Full	⚠️ Manual upload

Claude Code

100% compatible. Full pipeline support.

Cowork

~80% compatible. Loses bash, hooks, auto-load.

Claude.ai Projects

Manual use only. No pipeline execution.